

Potato

- 제한 시간: 10분
- 환경: GPU 1개(≈16 GB VRAM), 인터넷 없음
- 솔루션 크기: solution.ipynb ≤ 1 MB
- 저장 공간: 5 GB

과제

친구가 단어 맞추기 게임을 하자고 제안합니다. 친구는 심판으로서 고정된 어휘 집합에서 숨겨진 단어 하나를 고르며, 여러분은 최대 30턴 안에 그 단어를 찾아야 합니다. 매 턴마다 심판은 두 단어를 비교하여 어느 쪽이 숨겨진 단어에 의미적으로 더 가까운지 알려줍니다. 모든 게임은 고정된 쌍 lamp vs potato에서 시작하는데, 이는 여러분 친구가 가장 좋아하는 두 가지이기 때문입니다. 그다음 여러분이 만든 프로그램이 새로운 단어 하나를 제안합니다. 비교에서 이긴 단어는 유지되어 여러분의 다음 제안과 비교됩니다. 여러분이 숨겨진 단어를 정확히 제안하는 순간 그 게임에서 승리합니다. 일치 여부는 대소문자를 구분하지 않습니다. 여러분이 제안하는 모든 단어는 dataset/vocabulary.json에 있어야 합니다.

solution.ipynb에 프로토콜과 데이터 로딩을 포함한 완전한 예제가 있습니다. PublicEmbeddingPlayer 클래스를 변경할 수 있습니다. 여러분의 프로그램은 한번 초기화되어 단일 실행 안에서 모든 게임을 진행합니다. 프로토콜은 각 게임의 시작 시점에 새로운 PublicEmbeddingPlayer를 생성합니다.

심판(The Judge)

여러분의 프로그램은 심판에게 JSON 객체 하나를 보내고, 심판은 JSON 객체 하나로 응답합니다.

다음은 예시입니다. 숨겨진 단어는 예제로서, 프로토콜 (방법) 을 설명하기 위해서만 표시되어 있습니다:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}          <- matches: game won
-> {"status": "win"}
```

턴은 1부터 30까지의 인덱스를 가집니다.

verdict의 선택지는 word1이 더 가깝다는 의미의 first, word2가 더 가깝다는 의미의 second, 또는 두 단어가 숨겨진 단어에 똑같이 가깝다는 의미의 same입니다.

winner_word는 다음 비교를 위해 유지되는 단어입니다. same 판정의 경우, 첫 번째 단어가 유지됩니다.

데이터셋

모든 split이 공유하는 항목:

- dataset/vocabulary.json — 1602개의 고유한 소문자 단어. 숨겨진 단어는 항상 이들 중 하나입니다.
- dataset/public_embeddings.npy — float32, 형태(shape)는 (1602, 2560). 행 i는 어휘 집합의 단어 i에 대응합니다. 이들은 공개(public) 임베딩이며, 심판은 이와 다른 비공개 표현을 사용합니다.

split은 숨겨진 단어들의 집합입니다:

Split	단어 수	정답	용도
test_public	120	✓ dataset/test_public.json	솔루션 실행 및 자체 채점
test_leaderboard_a	120	비공개	실시간 리더보드
test_leaderboard_b	120	비공개	최종 순위

train split은 없습니다 — 레이블이 있는 행으로부터 학습(fitting)되는 것은 없습니다.

제공되는 모델

두 개의 사전학습된 임베딩 모델이 과제와 함께 제공되며 사용할 수 있습니다:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

둘 다 로컬 경로에서 로드해야 합니다. "BAAI/bge-m3"과 같은 Hugging Face hub id는 다운로드를 유발하여 실패하는데, 채점이 오프라인으로 이루어지기 때문입니다. 각 디렉터리에는 오프라인 호출 방법을 보여 주는 실행 가능한 `example.py`가 포함되어 있습니다.

사용 가능한 라이브러리: `numpy`, `torch`, `sentence-transformers`. 인터넷 없음, 다운로드 없음, 다른 패키지 없음.

출력

없음. 이는 인터랙티브 과제입니다. 여러분의 솔루션은 정답 파일을 작성하지 않으며, 위에서 설명한 대로 `stdin/stdout`을 통해 심판과 통신합니다.

평가 지표

턴 t 에서 찾아낸 게임은 $1.0 - 0.02 \times \max(0, t - 10)$ 점을 받고, 30턴 안에 풀지 못한 게임은 0점을 받습니다. 따라서 1-10턴은 1.00점, 20턴은 0.80점, 30턴은 0.60점입니다.

여러분의 과제 점수는 평균 게임 점수 $\times 100$ 이며, 0.00와 100.00 사이의 값입니다.

한 번 당 시작, 준비, 그리고 테스트 세트의 120 게임 전체를 수행하는데 10분의 시간 제한이 있습니다.

제출 방법

1. `solution.ipynb`를 열고, `PublicEmbeddingPlayer`를 편집한 뒤, 모든 셀을 실행하여 정상 작동하는지 확인하십시오.
2. 선택적으로, 로컬에서 확인하십시오: `python local_test.py solution.ipynb --limit 5`. 로컬 심판은 공개 임베딩을 사용하므로, 그 점수는 참고 용일 뿐입니다.
3. `solution.ipynb`을 저장하십시오.
4. JupyterLab의 왼쪽 사이드바에서 Git 탭을 여십시오.
5. `solution.ipynb`을 스테이징하십시오(옆에 있는 + 아이콘).
6. 커밋 메시지를 입력하고 Commit을 클릭하십시오.
7. 위쪽 화살표가 있는 구름 아이콘을 클릭하여 push하십시오.
8. 이 Contest 페이지로 돌아와, 여러분이 입력한 것과 동일한 커밋 메시지로 Submit을 클릭하십시오.

필요한 모든 준비 과정과 추론을 포함하여, `solution.ipynb`라는 이름의 파일 정확히 하나만 제출하십시오.